Assignment 5 solution

1. 【考点：I/O 管理基础】在计算机系统中，从早期的轮询（程序直接控制）到中断，再到 DMA（直接内存存取）控制方式，其主要发展目的是为了减少 CPU 对 I/O 过程的干预。关于 DMA 方式，下列说法正确的是（ ）

   A. DMA 控制器只能按字节为单位进行数据传送，无法处理连续的数据块
   B. DMA 方式下，I/O 设备与主存之间的数据交换必须经过 CPU 中的寄存器进行中转
   C. DMA 控制器在接管系统总线进行数据传送的过程中，不需要 CPU 参与，仅在数据块传送开始和结束时才需要 CPU 介入（产生中断）
   D. 为了提高传送效率，DMA 控制器在工作时总是会窃取 CPU 的指令周期来完成数据搬运

   【答案】：C
   【解析】：
   C 选项正确：DMA（Direct Memory Access）的核心特点就是数据在外设和内存之间直接传输，无需 CPU 执行程序指令干预。CPU 只需在初始化（指明源地址、目的地址和长度）和传送结束（处理 DMA 中断）时介入
   A 选项错误：DMA 传输的基本单位是数据块，而不是单字节
   B 选项错误：DMA 数据交换直接在“外设”和“主存”之间进行，不经过 CPU 寄存器
   D 选项错误：DMA 控制器窃取的是存储器周期（总线周期/机器周期），而不是指令周期。当 CPU 和 DMA 同时竞争总线时，通常由 DMA 优先使用（周期挪用/窃取）

2. 【考点：I/O 管理基础】现代计算机对 I/O 端口（设备控制器的寄存器）的编址方式主要分为独立编址（I/O 映射方式）和统一编址（内存映射 I/O）。关于统一编址（内存映射 I/O，Memory-Mapped I/O）的特点，下列说法正确的是（ ）

   A. 必须使用专门的硬件 I/O 指令（如 IN/OUT）才能访问设备的端口
   B. 占用了物理内存的地址空间，但 CPU 可以像访问普通内存变量一样，使用普通的取数/存数（如 MOV）指令直接控制外设
   C. 由于不需要额外的地址线，这种方式极大减少了主存的实际可用容量，因此仅适用于小型嵌入式系统
   D. 所有的字符设备都必须使用独立编址，而所有的块设备都必须使用统一编址

   【答案】：B
   【解析】：
   B 选项正确：内存映射 I/O 将 I/O 端口的地址映射到常规的主存地址空间中。CPU 对端口的访问等同于对特定内存地址的读写，因此直接使用普通的访存指令即可，灵活性高
   A 选项错误：需要专用 `IN/OUT` 指令的是独立编址（I/O 映射）
   C 选项错误：虽然占用了部分地址空间（虚拟/物理地址段），但现代计算机地址空间极大（如 64 位），这部分占用微不足道，广泛应用于几乎所有现代桌面/服务器架构
   D 选项错误：编址方式取决于 CPU 架构设计（如 x86 支持独立编址和内存映射，ARM/RISC-V 普遍只用内存映射），与设备属于字符设备还是块设备无关

3. 【考点：I/O 管理基础】操作系统中的 I/O 软件按层次分为了中断处理程序、设备驱动程序、设备独立性软件和用户层 I/O 软件。下列功能中，属于设备独立性软件（Device-Independent Software）负责完成的是（ ）

   A. 将逻辑设备名映射为物理设备名，并检查用户进程是否有权限访问该设备
   B. 直接操作硬件控制器的寄存器，启动具体的设备进行数据读写
   C. 在用户程序中调用 C 标准库函数 `printf()` 将字符串输出到终端
   D. 在硬件中断发生时保存当前进程的 CPU 上下文（断点和寄存器状态）

【答案】：A

【解析】：

A 选项正确：设备独立性软件（与设备无关的系统软件）主要负责向上层提供统一的接口、设备的分配与回收、缓冲区管理以及逻辑设备名到物理设备名的映射（通过 LUT 逻辑设备表）

B 选项错误：直接操作硬件寄存器是设备驱动程序的工作

C 选项错误：调用 `printf` 属于用户层 I/O 软件（库函数）

D 选项错误：保存硬件断点上下文是硬件及中断处理程序的工作

4. 【考点：I/O 管理基础】关于 I/O 应用程序接口中的阻塞 I/O（Blocking I/O）和非阻塞 I/O（Non-blocking I/O），下列说法错误的是（ ）

   A. 阻塞 I/O 系统调用发出后，若数据未准备就绪，调用进程将被操作系统挂起并转入阻塞态（等待态）

   B. 非阻塞 I/O 调用不论外设数据是否就绪，调用都会立即返回一个状态码，调用进程不会被挂起

   C. 使用非阻塞 I/O 时，用户进程通常需要通过循环轮询（Polling）来检查 I/O 操作是否真正完成，这可能导致 CPU 资源的浪费

   D. 异步 I/O（Asynchronous I/O）本质上就是非阻塞 I/O 的另一种称呼，二者的底层实现原理和执行流完全一致

   【答案】：D

   【解析】：

   D 选项错误（符合题意）：异步 I/O 与非阻塞 I/O 是完全不同的概念。非阻塞 I/O 需要进程主动去轮询检查状态，或者在数据就绪时由进程自己调用函数完成数据从内核态到用户态的拷贝；而异步 I/O（如 Windows IOCP 或 Linux io_uring）是指进程发起 I/O 请求后立即去做其他事，操作系统不仅负责等待数据就绪，还负责将数据拷贝到用户内存区，一切完成后再发送信号或回调函数通知用户进程

   A、B、C 选项的描述均完全符合这两种 I/O 模型的标准定义

5. 【考点：设备独立软件】为了缓和 CPU 与 I/O 设备之间速度不匹配的矛盾，操作系统广泛采用了缓冲池（Buffer Pool）机制。典型的缓冲池通常包含三个队列：空缓冲队列（emq）、输入队列（inq）和输出队列（outq）。当用户进程想要将计算好的数据输出到磁盘时，它在缓冲池中执行的正确操作步骤是（ ）

   A. 从 emq 队头取出一个空缓冲区，装满数据后，将其挂在 outq 队尾

   B. 从 inq 队头取出一个装满数据的缓冲区，读取数据后，将其挂在 emq 队尾

   C. 从 outq 队头取出一个装满数据的缓冲区，读取数据后，将其挂在 emq 队尾

   D. 从 emq 队头取出一个空缓冲区，装满数据后，将其挂在 inq 队尾

   【答案】：A

   【解析】：

   缓冲池管理中，数据的流转逻辑非常严谨：

   向外输出数据时（写操作）：进程需要找一个空的地方放数据，所以从“空缓冲队列 emq”中申请一个块 → 进程将数据写入缓冲区 → 写满后，这块就变成了待输出的脏数据，因此挂入“输出队列 outq”等待磁盘驱动程序将其写回外存。因此 A 正确

   选项 B 描述的是用户进程“读取输入数据”的过程

   选项 C 描述的是磁盘驱动程序将数据写到磁盘后的“回收”过程

   选项 D 是逻辑矛盾的

6. 【考点：设备独立软件】SPOOLing（Simultaneous Peripheral Operations On-Line）技术常用于将独占设备改造为共享设备（如网络打印机）。关于 SPOOLing 系统的组成与工作原理，下列说法正确的是（ ）

   A. SPOOLing 系统需要特定的专用通道硬件才能实现脱机输入输出，无法纯靠软件模拟

   B. 当用户进程请求打印时，SPOOLing 系统会立即分配物理打印机给该进程，直到打印任务结束才释放

   C. 预输入程序（输入进程）负责将用户进程所需的数据提前从内存输出到磁盘的输出井中

D. 在 SPOOLing 系统中，用户进程实际上是向磁盘上的“输出井”写入数据，该操作速度极快，使用户进程感觉独占了设备

【答案】：D
【解析】：
D 选项正确：SPOOLing 系统在高速磁盘上开辟了输入井和输出井。用户进程请求打印时，系统并没有真正分配打印机，而是在磁盘的“输出井”中为其申请了一块空闲区域将数据写入，并产生一份打印请求表。因为写磁盘速度远高于打印机，用户进程能迅速返回，在逻辑上实现了“设备共享”
A 选项错误：SPOOLing 正是用软件方式（多道程序设计技术）来模拟脱机 I/O，不需要专门的脱机硬件通道
B 选项错误：如果立即分配物理打印机，那就是传统的独占分配，会导致阻塞等待。SPOOLing 的核心是“假”脱机，拦截物理请求
C 选项错误：输入进程的作用是将数据从外部输入设备提取并写入输入井，而不是输出井

7. 【考点：外存管理】磁盘的读写时间主要由寻道时间、旋转延迟和传输时间三部分组成。为了优化旋转延迟时间，操作系统和磁盘控制器常常采用特定的扇区编排策略。假设某磁盘每磁道有 8 个扇区（编号 0~7），磁盘控制器读取一个扇区需要 1ms，处理读入的数据也需要 1ms，磁盘旋转一圈需要 8ms。若采用顺序扇区编号（即物理上相邻的扇区编号连续），且要连续读取该磁道上的所有 8 个扇区，则总耗时；若改为间隔 1 的交叉编号（Interleaving，即 0, 4, 1, 5, 2, 6, 3, 7），总耗时分别是?

   A. 16ms, 8ms
   B. 64ms, 16ms
   C. 64ms, 32ms
   D. 32ms, 16ms

   【答案】：B
   【解析】：
   顺序编号情况：读扇区0（1ms） → 此时磁盘继续转动，到达扇区1的起点 → CPU/控制器处理扇区0的数据需要 1ms。在这 1ms 期间，磁盘转过了扇区1（因为读一个扇区的时间正好是 1ms），等 CPU 处理完，磁头已经到了扇区2的起点。为了读扇区1，必须再等磁盘转一整圈（需要 7ms 重新绕回来）
   因此，读 1 个扇区并处理完的总周期是：读(1ms) + 错过扇区再转回来(7ms) + 处理(1ms但在这个转动期间已完成) = 读下一个扇区的准备间隔是 8ms。读 8 个扇区总耗时 = 8 × 8ms = 64ms（或者精确地说最后一次不需要等待转圈，为 57ms+读1ms=58ms，但题目估算总逻辑周期为 64ms）
   交叉编号情况：磁道物理排布为 0, 4, 1, 5, 2, 6... 读扇区0（1ms） → 磁盘转到物理上的扇区4 → CPU 处理数据（1ms），此时磁头刚好转过扇区4，到达扇区1的起点。CPU 刚好处理完，可以立即开始读扇区1
   因此，读+处理 1 个逻辑连续扇区的平均时间变成了 2ms。读 8 个扇区总耗时 = 8 × 2ms = 16ms
   由此可见，交叉编号极大消除了因处理速度跟不上而导致的额外旋转延迟

8. 【考点：外存管理】假设某磁盘共有 200 个柱面，编号为 0~199。当前磁头停在 100 号柱面，并且正向着磁道号增加的方向移动。此时有一个请求队列抵达：`[55, 58, 39, 18, 90, 160, 150, 38, 184]`。若分别采用 最短寻找时间优先（SSTF）算法 和 电梯调度（SCAN）算法 处理上述请求，磁头总的移动柱面数（即寻道距离之和）分别是（ ）

   A. SSTF 为 248；SCAN 为 248
   B. SSTF 为 248；SCAN 为 280
   C. SSTF 为 280；SCAN 为 280
   D. SSTF 为 280；SCAN 为 248

   【答案】：B
   【解析】：
   SSTF 算法（每次选择离当前磁头最近的请求）：
   起点：100 → 最近的是 90（移 10）

90 → 最近 58（移 32）
58 → 最近 55（移 3）
55 → 39（移 16）
39 → 38（移 1）
38 → 18（移 20）
此时较小方向走完了，只剩大的：18 → 150（移 132）
150 → 160（移 10）
160 → 184（移 24）
SSTF 总移动数 = 10 + 32 + 3 + 16 + 1 + 20 + 132 + 10 + 24 = 248

SCAN 算法（电梯算法：沿着当前方向走到尽头请求处，再折返）：
当前方向：朝磁道号增加的方向
移动轨迹与计算：
第一段（向外侧移动至边界）： 从 100 开始，依次处理 150, 160, 184。注意，处理完 184 后不能直接折返，必须继续走到该方向的物理尽头，即 199 号柱面
移动距离 = 199 - 100 = 99
第二段（折返向内侧移动）： 从 199 掉头，依次处理 90, 58, 55, 39, 38, 18
移动距离 = 199 - 18 = 181
SCAN 总移动数： 99 + 181 = 280

9. 【考点：外存管理】固态硬盘（SSD）凭借极高的 I/O 吞吐量逐渐取代了传统机械硬盘。关于 NAND Flash 固态硬盘底层的读写物理特性与机制，下列说法正确的是（ ）

A. SSD 的最小读写和擦除单位都是“页（Page）”，通常大小为 4KB 到 16KB 之间
B. SSD 存在“写放大（Write Amplification）”现象，是因为其数据写入必须遵循“先擦除，后写入”的规则，且擦除的最小物理单位是“块（Block）”，它包含了几十到成百上千个“页”
C. 操作系统向 SSD 的某个逻辑块号（LBA）重复写入新数据时，SSD 控制器会直接在原物理页上进行覆盖写入操作
D. 为了延长 SSD 寿命，应在操作系统中开启定期的磁盘碎片整理服务（Defragmentation）

【答案】：B
【解析】：
B 选项正确：SSD 的一大物理特性是写放大。因为 SSD 无法就地覆盖修改数据。如果要修改某页的数据，由于只能在空白页写入，且擦除的单位必须是巨大的一整个“块（Block）”，这就需要将该块中其他还有用的页搬运到新块中，再擦除旧块，从而导致实际写入量大于逻辑写入量
A 选项错误：最小读、写单位是“页（Page）”，但最小擦除单位是“块（Block）”
C 选项错误：由于不支持原址覆盖写（Overwrite），SSD 会将新数据写在一个全新的空闲页上，并通过闪存转换层（FTL）将该逻辑块号映射到新的物理页，同时将老物理页标记为“失效（Invalid/Stale）”
D 选项错误：磁盘碎片整理会产生大量的无效擦写，极大损耗闪存颗粒的寿命（P/E 次数），且由于 SSD 的寻址是纯电信号（无需磁头机械寻道），碎片整理对读取速度毫无增益。操作系统对 SSD 应开启 TRIM 指令而非碎片整理

10. 【考点：外存管理】由于固态硬盘（SSD）的每个闪存块（Block）都有物理上的最大擦写次数限制（P/E Cycles），现代 SSD 控制器内部都会部署“磨损均衡（Wear Leveling）”算法以延长整个硬盘的使用寿命。关于磨损均衡，以下描述错误的是（ ）

A. 动态磨损均衡（Dynamic Wear Leveling）是在系统写入新数据时，优先挑选擦除次数较少（即较“新”）的空闲块来承载写入操作
B. 纯粹的动态磨损均衡存在缺陷：如果一块数据写入后变成了冷数据（长期不被修改，如操作系统核心文件），它占据的那个物理块的擦写次数将永远停滞在低水平，导致硬盘各块磨损严重不均
C. 静态磨损均衡（Static Wear Leveling）会定期扫描那些很久没有发生过擦写的冷数据块，将其中的数据搬迁到擦写次数较高的块中，从而把低擦写次数的块腾出来用于日常高频写入
D. 为了降低系统的开销，静态磨损均衡通常被设计在操作系统的文件系统层实现，而不是做在 SSD 控制器的固件中

【答案】：D
【解析】：
D 选项说法错误（符合题意）：磨损均衡无论是静态还是动态，均属于闪存转换层（FTL，Flash Translation Layer）的核心固件算法，被固化在 SSD 控制器硬件内部的微码中。操作系统对此是完全透明的（即操作系统根本不知道底层是哪个物理块被擦除），操作系统只管逻辑区块地址（LBA）的读写
A、B、C 选项准确描述了动态与静态磨损均衡的机理。静态磨损均衡通过主动搬迁“钉子户”（冷数据）来迫使整个盘所有闪存颗粒的寿命同步消耗，是高端 SSD 的标配

11. 【考点：I/O控制方式（轮询方式，中断方式，DMA方式）】在现代高性能服务器（如处理 10Gbps 网络流量或 NVMe 存储）中，CPU 与外设之间的数据交换效率是系统的核心瓶颈。关于轮询、中断和 DMA（直接内存访问）三种 I/O 控制方式，下列说法完全正确的是（ ）

    A. 纯轮询（Polling）方式会极大浪费 CPU 周期，因此在任何现代高性能网络处理框架（如 DPDK）中，都已经彻底废弃了轮询机制，全面转向纯中断驱动
    B. 在中断驱动 I/O 方式中，设备每完成一个“数据块”的传输就会触发一次中断。由于其传输单位是块，因此特别适合海量数据的磁盘 I/O 操作
    C. 在 DMA 方式下，CPU 只需在数据传输开始时配置源地址、目的地址和传输长度，实际的内存与外设间数据搬运由 DMA 控制器完成。传输结束后，DMA 控制器通过中断通知 CPU
    D. 当 DMA 控制器接管系统总线进行数据传输时，它与 CPU 在硬件物理上是完全并行工作的。因此，DMA 传输不会对 CPU 从主存中预取指令或读写数据产生任何哪怕极其短暂的阻塞影响

    【答案】： C
    【解析】：
    C 选项完全正确：DMA（Direct Memory Access）的核心思想就是“解放 CPU”。CPU 仅负责初始化（下发传输指令参数）和收尾（处理传输完成中断），中间繁重的数据字/字节跨总线搬运工作全部交由专门的硬件（DMA 控制器）独立完成
    A 选项错误：在极高负载的高性能网络场景（如 NAPI 机制或 DPDK 框架）下，频繁的中断会导致严重的上下文切换开销（中断风暴）。此时，工程上反而会关闭中断，主动采用轮询（Polling）方式去网卡缓冲区拉取数据，以榨干 CPU 性能换取极致的低延迟和高吞吐
    B 选项错误：中断驱动 I/O（Interrupt-driven I/O）的基本传输单位通常是字（Word）或字节（Byte），而不是“块”。每传输一个字都要引发一次中断，对于高速大容量的磁盘 I/O 来说，这种级别的中断频率是 CPU 无法承受的，必须使用 DMA
    D 选项错误：由于 DMA 控制器和 CPU 共享系统的内存总线，当 DMA 正在向主存高速写入数据时，如果 CPU 也需要访问主存（如 Fetch 指令），DMA 会强制 CPU 暂停一个或几个主存访问周期，这在计算机体系结构中被称为“周期挪用”或“周期窃取”（Cycle Stealing）。因此，不可能“没有任何影响”

12. 【考点：I/O 软件层次结构】操作系统 I/O 软件的层次化设计类似于网络协议栈，每一层屏蔽底层的复杂性并向上提供抽象接口。关于 I/O 软件层次结构（从用户层到硬件）及其对应的应用程序接口，下列说法完全正确的是（ ）

    A. 在 Linux 系统中，键盘和鼠标因为输入数据具有随机性和离散性，被归类为块设备接口（Block Device Interface），而硬盘则属于字符设备接口
    B. 设备独立软件（Device-Independent Software）层负责完成统一的设备命名、逻辑设备到物理设备的映射、缓冲区管理以及向硬件下发具体的读写寄存器指令
    C. 中断处理程序（Interrupt Handlers）是 I/O 层次结构中最底层的软件模块。当 I/O 完成硬件中断发生时，中断处理程序通常会唤醒由于等待该 I/O 而被阻塞的设备驱动程序进程
    D. 若应用程序以非阻塞 I/O（Non-blocking I/O）模式调用 `read()` 读取网络套接字，当网卡缓冲区尚无数据时，操作系统内核会自动在后台创建一个守护线程等待数据，直到数据到达后再推送给应用程序

    【答案】： C
    【解析】：
    C 选项完全正确：在 I/O 软件体系中，中断处理程序是最贴近硬件的一层软件。设备驱动程序在启动硬件后通常会自我阻塞（进入睡眠等待态）。当设备完成操作触发硬件中断后，中断处理程序负

责保存现场、分析中断原因，并唤醒处于睡眠态的相应驱动程序去处理读取到的数据
A 选项错误：描述完全反了。键盘、鼠标等不可寻址、以字节流方式顺序访问的设备属于字符设备。而硬盘这种支持随机寻址、以固定大小的数据块（如 4KB）为传输单位的设备属于块设备
B 选项错误：设备独立软件（或称设备无关软件）确实负责统一命名、映射和缓冲区管理等通用抽象功能，但它绝对不会直接触碰硬件寄存器。向底层特定硬件控制器下发读写指令，是设备驱动程序（Device Driver）的专属核心职责
D 选项错误：非阻塞 I/O（如设置了 `O_NONBLOCK` 标志）的语义是：如果数据没准备好，`read()` 系统调用会立刻返回一个错误码（如 `EWOULDBLOCK` 或 `EAGAIN`），告诉进程"现在没数据，请稍后再试"。它绝不会在内核自动创建后台线程去推数据（那是异步 I/O 或信号驱动 I/O 的范畴）

13. 【考点：设备分配与回收】在早期批处理系统以及现代云打印服务中，独占型设备极易成为拖慢系统整体吞吐量的瓶颈。为了解决这一问题，操作系统引入了 SPOOLing（假脱机）技术。关于设备分配策略与 SPOOLing 技术，下列描述完全正确的是（ ）

    A. SPOOLing 技术是一种纯硬件层面的加速技术，它在 CPU 和慢速外设之间增加了一块高速 SRAM 作为专属缓存，从而彻底消除了速度不匹配的问题
    B. 借助 SPOOLing 技术，原本只能被一个进程独占使用的设备（如打印机）在逻辑上被改造为共享设备。多个用户进程可以并发地将打印输出请求写入磁盘队列中，而无需等待物理打印机空闲
    C. 虚拟设备技术（SPOOLing）的实现仅仅依赖于内存中的输入/输出缓冲区，不需要任何外部磁盘存储空间（外存）的支持
    D. 在操作系统的设备分配中，由于 SPOOLing 将独占设备虚拟化了，因此分配此类虚拟设备时，操作系统永远不需要考虑死锁（Deadlock）的预防或避免问题

    【答案】： B
    【解析】：
    B 选项完全正确：SPOOLing（Simultaneous Peripheral Operations On-Line，外部设备联机并行操作）的本质就是"将独占设备虚拟化为共享设备"。当多个进程请求打印时，SPOOLing 系统会接管请求，将数据暂存到磁盘的输出井中，并对进程伪装成"已经打印完毕"。然后，后台守护进程按照队列顺序慢慢将磁盘上的数据送入物理打印机。这允许进程不被物理外设阻塞
    A 选项错误：SPOOLing 是一项纯软件技术（依赖操作系统守护进程和磁盘抽象），而不是硬件 SRAM 缓存技术
    C 选项错误：SPOOLing 技术的实现高度依赖于大容量的外存（如磁盘）。它的核心结构就是在磁盘上开辟的"输入井"和"输出井"，用来持久化暂存进程的海量 I/O 数据。仅靠有限的内存缓冲区是无法承载大量并发任务的假脱机输出的
    D 选项错误：虽然 SPOOLing 缓解了独占设备的物理争用，但由于磁盘上的输入/输出井空间（即 Spooling 队列容量）是有限的，如果大量进程疯狂涌入并填满了磁盘的输出井空间，依然可能导致进程在等待输出井释放空间时发生"死锁（Deadlock）"现象

14. 【考点：磁盘结构，格式化，磁盘调度方法】在数据库存储引擎底层的性能调优中，针对机械磁盘的 I/O 调度是至关重要的一环。机械臂的物理寻道时间构成了 I/O 延迟的主要开销。关于磁盘物理结构与磁盘调度算法，下列说法完全正确的是（ ）

    A. 在磁盘发生单次读写操作的总时间里，由于盘片旋转产生的"旋转延迟时间（Rotational Latency）"通常是最耗时的部分，远大于磁头移动的"寻道时间（Seek Time）"
    B. 磁盘的低级格式化（物理格式化）主要任务是创建操作系统的文件系统，如划分目录树和创建 inode 表；而高级格式化（逻辑格式化）则是划分扇区和磁道
    C. 最短寻道时间优先（SSTF）算法由于每次总是选择距离当前磁头最近的请求进行服务，因此它是所有调度算法中平均寻道时间最短的，且绝对不会产生"饥饿"现象
    D. 扫描算法（SCAN，又称电梯算法）要求磁头臂必须单向移动，直到处理完该方向上的所有请求并触及磁盘的物理边界（或最边缘请求）后，才会反转方向。这有效克服了 SSTF 算法的"饥饿"缺陷

    【答案】： D
    【解析】：
    D 选项完全正确：SCAN 算法模拟电梯运行，强制磁头臂保持一个方向移动，沿途处理所有请求，直到遇到边界（严格的 SCAN 到达物理边缘，其变种 LOOK 到达最远请求即可），然后掉头。这

种强制轮询的机制确保了即使某一区域有密集的局部 I/O 请求，远端的请求也必然能在磁头的一趟完整扫描中被服务，从而根除了 SSTF 的饥饿问题

A 选项错误：磁盘 I/O 时间 = 寻道时间 + 旋转延迟 + 传输时间。其中，寻道时间（机械臂径向移动找磁道）通常是最耗时的（毫秒级别，约 4~10ms）。旋转延迟次之（通常是半圈的时间，7200转硬盘约 4.17ms）

B 选项错误：格式化的概念颠倒了。低级格式化是出厂前或底层硬件操作，负责在空白盘片上刻画出磁道和扇区的物理结构。高级格式化才是操作系统的行为，负责在分区上创建文件系统、逻辑块结构、根目录和 inode 数据

C 选项错误：SSTF 算法存在严重的“饥饿（Starvation）”问题。如果某个区域（如内圈）持续不断地产生新的近距离 I/O 请求，磁头会在该区域“流连忘返”，导致远端（如外圈）的早期请求被无限期搁置

15. 【考点：固态硬盘读写性能特性，磨损均衡】现代云计算存储底座几乎全部迁移至固态硬盘（SSD），其基于 NAND 闪存的读写特性彻底颠覆了传统的磁盘管理思维。关于 SSD 的物理读写性能特性及磨损均衡（Wear Leveling）策略，下列描述完全正确的是（ ）

A. SSD 在进行数据覆盖写入时，与机械硬盘类似，可以直接在包含旧数据的物理闪存页（Page）上进行原地覆盖（In-place Overwrite），从而保证极高的写入速度

B. SSD 的物理基础限制决定了其数据的“读”和“写”是以“页（Page）”为单位的，但“擦除（Erase）”操作却必须以更大的“块（Block）”为单位进行

C. 静态磨损均衡（Static Wear Leveling）策略仅在写入新数据时，选择擦除次数较少的空闲块。对于长期存放操作系统核心只读文件（冷数据）的物理块，该策略从不干预

D. 由于 SSD 没有机械寻道时间，因此操作系统的磁盘调度算法（如 SCAN 或 C-SCAN 算法）可以直接且有效地提升 SSD 的随机 I/O 吞吐率，其原理与在 HDD 上完全一致

【答案】： B

【解析】：

B 选项完全正确：这是 NAND 闪存最核心的物理限制。SSD 中一个“块（Block）”包含多个“页（Page）”。数据的读取和编程（写入）可以在“页”级别（通常 4KB 或 8KB）进行。但当需要清理旧数据时，由于闪存电路结构的限制，擦除操作只能在包含数十到数百个页的庞大“块（Block）”级别集体进行

A 选项错误：SSD 绝对不支持“原地覆盖（In-place Overwrite）”。由于“擦除前写（Erase-before-write）”的特性，如果某个页已有数据，必须先将整个块擦除才能再次写入该页。工程上，SSD 的控制器（FTL）会采取异地更新（Out-of-place update）：将新数据写到一个干净的新页中，并将原来的旧页标记为“失效/垃圾”，随后通过垃圾回收（GC）统一擦除

C 选项错误：描述的是动态磨损均衡的缺点。真正的静态磨损均衡（Static Wear Leveling）更进一步：它会主动在后台将那些长期未被修改的“冷数据”（如 OS 内核文件）从健康良好的物理块中搬迁出来，存放到擦除次数较高的旧块中，从而把完好、寿命长的新块腾出来给频繁覆写的数据使用。静态均衡对提升 SSD 整体寿命最为关键

D 选项错误：SSD 没有物理磁头和旋转盘片，其任意物理地址的访问延迟几乎是恒定的。像 SCAN（电梯算法）这类专为减少机械寻道距离而设计的调度算法对 SSD 毫无意义。在 SSD 环境中，操作系统通常采用 NOOP（无操作/简单的 FIFO 队列）或特定的 Deadline 调度器，甚至利用 NVMe 接口的多队列（Multi-queue）并发直通特性

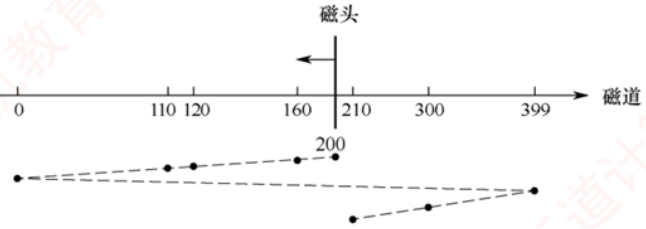
16. **46．C**

以 C 语言中 scanf()函数的执行过程为例，用户请求通过键盘输入数据，当程序执行到 scanf()时，触发系统调用，CPU 切换到内核态，执行系统调用服务例程，系统调用的内核程序进行一些初始化操作，启动外设，同时阻塞该用户进程，直到键盘输入数据，键盘中断服务例程（实际上是驱动程序）将数据从 I/O 接口（键盘控制器）中的数据寄存器取出并送至内核缓冲区，然后唤醒用户进程，用户进程被唤醒后进入就绪队列，等得到 CPU 运行后，再将内核缓冲区中的数据送至用户缓冲区，最后进行系统调用的返回。综上所述，键盘中断服务例程执行结束后，所输入数据的存放位置是内核缓冲区，当系统调用结束时，所输入数据才被送至用户缓冲区。

17.

34. C

采用 C-SCAN 算法进行磁盘调度，完成 200 号磁道的请求后，磁头向磁道号减小的方向移动，依次处理 160, 120, 110, 0 号磁道的请求；然后，磁头向磁道号增大的方向快速移动至 399 号磁道，在快速移动的过程中不处理任何请求；完成 399 号磁道的请求后，依次处理 300, 210 号磁道的请求，所以磁头移动的总距离是(200 – 0) + (399 – 0) + (399 – 210) = 788。调度过程如下图所示。

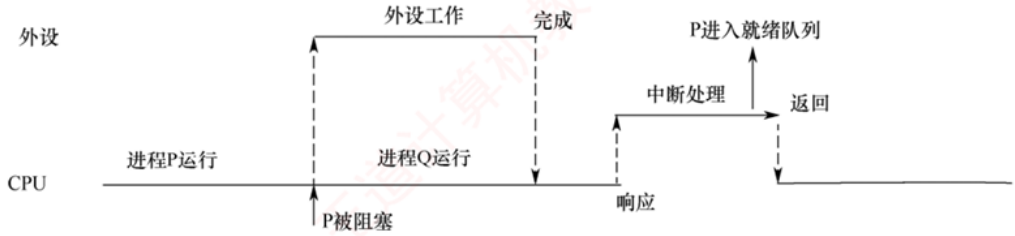


18. **35**. B

无论是温彻斯特硬盘（机械磁盘）还是固态硬盘，文件系统均以固定大小的逻辑盘块为单位组织和管理存储空间，因此**确定盘块大小**是两者都必须提供的基础功能。相比之下，“划分扇区”是机械硬盘固件或硬件层面的操作，文件系统无须参与，固态硬盘无传统扇区结构；“降低寻道时间”仅针对存在磁头移动的机械硬盘，固态硬盘无机械部件，无寻道问题；“均衡磨损”则是固态硬盘为延长闪存寿命所特有的机制，机械硬盘无须考虑。

19. **03**. 【解答】

1）正确的执行顺序为②⑥④③①⑤，因此操作①的前一个操作是③，后一个操作是⑤；操作⑥的后一个操作是④。下面以 scanf()函数的执行过程为例分析相关操作顺序。首先用户进程 P 调用 I/O 标准库的 scanf()函数，scanf()调用系统调用封装函数 read()，在这个函数中有一条陷阱指令，通过它陷入内核，内核调出 read()对应的 sys_read()系统调用服务例程进行执行，进入内核后，在设备无关层进行若干调用，最终到设备驱动层进行处理，设备驱动程序对本次 I/O 进行初始化操作，然后执行操作系统的进程调度程序 scheduler()，由调度程序 scheduler()来阻塞用户进程 P，并调度其他进程执行。此后，在用户输入字符之前，CPU 运行其他进程，键盘等待用户输入数据，当用户在键盘上输入字符后，外设向 CPU 发出相应的中断请求，CPU 响应中断后，启动键盘中断处理程序，该处理程序对 I/O 中断进行简单通用的处理后，唤醒具体的处理键盘输入的驱动程序，该驱动程序将字符从键盘控制器读入系统缓冲区，数据传输完成后，唤醒 P 并将其插入就绪队列，进行中断返回，进程 P 也从系统调用返回。相关过程如下图所示。



2）在操作②之后 CPU 一定从进程 P 切换到其他进程。在操作①之后 CPU 调度程序才能选中进程 P 执行。

3）设备驱动程序负责驱动 I/O 设备工作，I/O 操作初始化，执行具体的 I/O 指令。将字符从键盘控制器读入系统缓冲区是和键盘直接相关的具体操作，完成操作③的代码属于键盘驱动程序。

4）键盘中断处理程序执行时，进程 P 还在阻塞队列，处于阻塞态。中断处理程序、设备驱动程序、设备独立性软件都属于内核 I/O 软件层，执行相关代码时，CPU 处于内核态。

20. **06**. 【解答】

1）用位图表示磁盘的空闲状态。每位表示一个磁盘块的空闲状态，共需 16384/32 = 512 个

字 ＝512×4B＝2KB，正好可放在系统提供的内存中。

2）采用 C-SCAN 算法，访问磁道的顺序和移动的磁道数如下表所示：

被访问的下一个磁道号	移动距离（磁道数）
120	20
30	90
50	20
90	40

移动的磁道数为 20 + 90 + 20 + 40 = 170，因此总的移动磁道时间为 170ms。

转速为 6000 转/分，因此平均旋转延迟为 5ms，总的旋转延迟时间 ＝20ms。

转速为 6000 转/分，因此读取一个磁道上的一个扇区的平均读取时间为 0.1ms，扇区的平均读取时间为 0.1ms，总的读取扇区的时间为 0.4ms。

综上，读取上述磁道上所有扇区所花的总时间为 190.4ms。

3）采用先来先服务（FCFS）调度策略更高效。因为 Flash 半导体存储器的物理结构不需要考虑寻道时间和旋转延迟，可直接按 I/O 请求的先后顺序服务。

21. **07.【解答】**

1）磁盘容量 ＝ 磁盘的柱面数×每个柱面的磁道数×每个磁道的扇区数×每个扇区的大小

＝(300×10×200×512/1024) KB = 3×10^5KB。

2）磁头在 85 号柱面上，对 SSTF 算法而言，总是访问当前柱面距离最近的地址。注意每个簇包含 2 个扇区，通过计算得到，85 号柱面对应的簇号为 85000～85999。通过比较得出，系统最先访问离 85000～85999 最近的 100260，随后访问离 100260 最近的 101660，然后访问 110560，最后访问 60005。顺序为 100260，101660，110560，60005。

3）第 100530 簇在磁盘上的物理地址由其所在的柱面号、磁头号、扇区号构成。

柱面号 ＝⌊簇号/每个柱面的簇数⌋ = ⌊100530/(10×200/2)⌋ ＝ 100。

磁头号 ＝⌊(簇号%每个柱面的簇数)/每个磁道的簇数⌋ = ⌊530/(200/2)⌋ ＝ 5。

扇区号 ＝ 扇区地址%每个磁道的扇区数 ＝ (530×2)%200 ＝ 60。

将簇号转换成磁盘物理地址的过程由磁盘驱动程序完成。

22. **08.【解答】**

1）执行顺序依次是 ROM 中的引导程序、磁盘引导程序、分区引导程序、操作系统的初始化程序。启动系统时，首先运行 ROM 中的引导代码（bootstrap）。为执行某个分区的操作系统的初始化程序，需要先执行磁盘引导程序以指示引导到哪个分区，然后执行该分区的引导程序，用于引导该分区的操作系统。

2）4 个操作的执行顺序依次是磁盘的物理格式化、对磁盘进行分区、逻辑格式化、操作系统的安装。磁盘只有通过分区和逻辑格式化后才能安装系统和存储信息。物理格式化（也称低级格式化，通常出厂时就已完成）的作用是为每个磁道划分扇区，安排扇区在磁道中的排列顺序，并对已损坏的磁道和扇区做“坏”标记等。随后将磁盘的整体存储空间划分为相互独立的多个分区（如 Windows 中划分 C 盘、D 盘等），这些分区可以用作多种用途，如安装不同的操作系统和应用程序、存储文件等。然后进行逻辑格式化（也称高级格式化），其作用是对扇区进行逻辑编号，建立逻辑盘的引导记录、文件分配表、文件目录表和数据区等。最后才是操作系统的安装。

3）由上述分析可知，磁盘扇区的划分是在磁盘的物理格式化操作中完成的，文件系统根目录的建立是在逻辑格式化操作中完成的。

23. 【考点：I/O 软件层次结构】关于 I/O 控制方式、I/O 软件层次结构及中断处理机制，下列说法中正确的是（ ）

A. 在 DMA（直接内存访问）数据传输阶段，CPU 仍然需要通过轮询方式逐个字节地读取外设端口并将数据搬运至内存中

B. 设备驱动程序属于设备独立软件层，其核心任务是将文件系统的逻辑块号直接转换为磁盘表面的物理柱面、磁头和扇区号

C. 中断处理程序作为 I/O 软件的底层，其主要任务是保存现场、识别中断源并清除中断标志，通常由其直接负责将阻塞的应用程序彻底唤醒并投入就绪队列

D. 缓冲池（Buffer Pool）管理机制属于设备独立软件层的范畴，系统通过维护空缓冲队列、输入队列和输出队列，实现数据的高效暂存与解耦

【答案】： D

【解析】：

A 项错误：在 DMA 方式下，数据的传输是由 DMA 控制器直接在I/O设备与内存之间进行的，整个传输阶段不需要 CPU 逐个字节参与轮询搬运

B 项错误：将逻辑块号转换为物理盘块（柱面、磁头、扇区）的职责属于设备独立软件层（或文件系统相关层），而设备驱动程序是紧密贴合硬件的底层代码，负责直接向设备控制器寄存器发送命令

C 项错误：中断处理程序运行在严苛的中断上下文中，其主要职责是快速响应硬件、清除中断标志、保存底层硬件现场，并触发底半部（Bottom Half）处理，不能直接安全地执行复杂的进程唤醒和调度逻辑

D 项正确：缓冲区管理（如单缓冲、双缓冲、缓冲池）属于设备独立软件的核心功能，通过在内存中维护多个缓冲队列，有效平缓了 I/O 速度与 CPU 处理速度的矛盾

24. 【考点：假脱机技术】假脱机技术（SPOOLing）是操作系统中实现设备共享的核心技术。关于 SPOOLing 技术、设备分配与虚拟化，下列说法中错误的是（ ）

A. SPOOLing 技术在磁盘上开辟输入井和输出井，在内存中开辟输入缓冲区和输出缓冲区

B. 引入 SPOOLing 技术后，用户进程申请打印输出时，系统并没有把真实的物理打印机直接分配给该进程，而是将输出数据先存放在磁盘输出井中，从而伪造出一个虚拟打印机

C. 独占设备如果采用 SPOOLing 技术进行管理，虽然多个进程可以“同时”向虚拟设备提交请求，但宏观上这些任务最终仍需由守护进程串行地输出到真实的物理设备上

D. SPOOLing 技术主要依赖多道程序设计技术和通道技术，将一台独占设备彻底改造为可以同时进行原地物理修改的共享存储设备

【答案】： D

【解析】：

A、B、C 项正确：准确描述了 SPOOLing 的空间分配原则（磁盘作井、内存作缓冲区）、虚拟设备分配机制（假分配）以及对独占设备（如打印机）的共享转换流程

D 项错误：SPOOLing 技术是通过共享盘空间（输入/输出井）将独占设备虚拟化为多台逻辑上的虚拟设备，使其能够以批处理方式并发服务，并没有将其改造为“可进行原地物理修改的共享存储设备”（例如打印机本质是输出设备，不支持随机的原地修改）

25. 【考点：磁盘调度算法】磁盘和固态硬盘（SSD）作为常见的两类外存设备，其底层调度与读写特性存在显著差异。下列关于外存管理机制的叙述中，正确的是（ ）

A. 在传统机械磁盘调度算法中，C-SCAN（循环扫描）算法相比于 SCAN 算法，虽然消除了磁头移动的“偏向性”，但其平均寻道时间一定比 SCAN 算法更长

B. 固态硬盘（SSD）由于内部采用闪存芯片，支持原位修改（In-place update）机制，因此其随机写入性能与随机读取性能完全对称

C. SSD 内部由于“擦除以块为单位、读写以页为单位”的硬件物理约束，必须依赖闪存转换层（FTL）来实现地址映射、垃圾回收（Garbage Collection）和磨损均衡

D. 动态磨损均衡（Dynamic Wear Leveling）算法完全由操作系统内核的文件系统层实现，不需要 SSD 硬件控制器或 FTL 的参与

【答案】： C

【解析】：

A 项错误：C-SCAN 算法每次单向扫描后直接快速返回起点，其平均寻道时间不一定比 SCAN 长，具体取决于访问请求的分布情况

B 项错误：闪存的物理特性决定了它不支持原位修改（必须先擦除整块才能写入），因此 SSD 必须通过“异地更新（Out-of-place update）”配合垃圾回收机制来实现写入，其随机写性能通常远低于随机读性能

C 项正确：由于闪存“擦除块远大于读写页”的硬件物理限制，必须通过闪存转换层（FTL）进行逻辑地址到物理地址的映射转换、垃圾回收及磨损均衡

D 项错误：磨损均衡（Wear Leveling）的核心管理逻辑是由 SSD 内部的 FTL 固件和硬件控制器来完成的，而不是完全依赖操作系统的文件系统层

26. 【考点：输入/输出应用程序接口】在操作系统对 I/O 应用程序接口及设备独立软件的设计中，阻塞与非阻塞 I/O 以及设备分配策略密切相关。下列关于 I/O 接口与设备分配的叙述中，错误的是（ ）

A. 字符设备接口通常以字节流为单位进行数据传输，支持依次读写，一般采用字节缓冲或直接无缓冲交互
B. 块设备接口通常以数据块（如 512 字节或 4KB）为单位进行传输，支持随机寻址，因此块设备接口通常配合磁盘缓存机制使用
C. 采用非阻塞 I/O 调用的进程在发起 I/O 请求后，若数据未准备好，系统会立即让出 CPU 并将该进程阻塞挂起，直到数据传输完成后再由中断唤醒
D. 设备独立软件层负责设备的分配与回收，在分配独占设备时，通常采用安全性检查或静态预分配策略来防止系统发生死锁

【答案】： C
【解析】：
A、B、D 项正确：分别准确概括了字符设备/块设备接口的读写特性、磁盘缓存的必要性，以及设备独立软件层在独占设备分配中为防止死锁所采取的静态预分配/安全性检查策略
C 项错误：此题混淆了阻塞 I/O 与非阻塞 I/O 的概念。非阻塞 I/O 调用的核心特点是：若外设数据未准备好，系统会立即返回一个错误状态码（如 `-EAGAIN`），而不会将进程阻塞挂起；只有阻塞 I/O 才会让出 CPU 并将进程挂起

27. 【考点：I/O 软件层次结构】关于 I/O 控制方式、中断处理机制及设备独立软件层的协同工作，下列说法中正确的是（ ）

A. DMA 传输结束触发的中断属于外中断，CPU 响应时由硬件自动完成关中断，并必然强制引发从用户态到内核态的特权级切换
B. 中断处理程序的顶半部（Top Half）可以直接执行复杂的进程唤醒、阻塞队列调整和高层调度算法，以确保高优先级 I/O 任务的实时响应
C. 缓冲区管理（如缓冲池机制）属于设备独立软件层，当 DMA 传输完成后，内核通过调整缓冲区状态，配合设备驱动程序向上层提供数据传输服务
D. 在 DMA 数据传输的整个生命周期中，CPU 必须持续处于休眠或挂起状态，直到硬件发出中断信号前一刻才能恢复执行

【答案】： C
【解析】：
A 项错误：DMA 中断属于外中断，硬件会自动关中断并保存现场，但若触发中断时 CPU 已经在内核态执行，则不会发生用户态到内核态的特权级切换
B 项错误：中断处理程序的顶半部运行在严格的中断上下文中，严禁执行可能导致睡眠、阻塞或复杂调度的操作（如进程唤醒与队列调整），这些工作必须推迟到底半部（Bottom Half）或软中断中处理
C 项正确：缓冲区管理属于操作系统的设备独立软件层，通过管理输入/输出队列与缓冲区状态，有效解耦了高速 CPU 与低速外设
D 项错误：DMA 方式的核心优势在于数据搬运由 DMA 控制器独立完成，期间 CPU 可以并发执行其他进程的指令，而不是持续处于休眠状态

28. 【考点：磁盘调度算法】某系统采用改进的 LOOK 磁盘调度算法处理来自不同进程的磁盘 I/O 请求队列。假设磁盘磁道总数为 200（编号 0~199），当前磁头位于 100 号磁道，且正向磁道号增大的方向移动。现有请求队列包含对下列磁道的访问请求：125, 40, 180, 20, 150。若严格按照 LOOK 算法执行（规定磁头在当前方向无请求时立即折返，且只移动到最远请求处），下列关于该调度过程的叙述中，错误的是（ ）

A. 磁头首先按原方向依次访问 125、150、180 号磁道，到达当前方向最远请求 180 号磁道后立即改变移动方向
B. 折返后，磁头向磁道号减小的方向移动，依次访问 40 号和 20 号磁道
C. 在整个 I/O 请求队列处理完毕的过程中，磁头移动的总磁道数（寻道距离）为 160
D. 磁盘调度算法通常由操作系统的设备驱动程序或设备独立软件层实现，其核心目标是最小化平均寻道时间以提升外存吞吐量

【答案】： C
【解析】：
A、B 项正确：根据 LOOK 算法规则，磁头从 100 出发，向增大方向前进，依次经过 125、150、到达最远请求 180；随后无同方向请求，立即折返向减小方向移动，依次经过 40、20
C 项错误（计算验证）：寻道路径为 100 → 125 → 150 → 180 → 40 → 20
各段距离计算：| 125-100 | + | 150-125 | + | 180-150 | + | 40-180 | + | 20-40 | = 25 + 25 + 30 + 140 + 20 = 240 磁道
题目中称总磁道数为 160 计算错误，故 C 项错误，符合题意
D 项正确：磁盘调度算法属于操作系统底层设备管理范畴，旨在减少平均寻道时间

29. 【考点：固态硬盘】固态硬盘（SSD）与传统机械磁盘在物理架构及管理机制上存在本质区别。下列关于 SSD 的读写性能特性、闪存转换层（FTL）及磨损均衡的叙述中，正确的是（ ）

A. SSD 内部的闪存芯片支持原位修改（In-place update），因此其随机写入性能与随机读取性能完全对称
B. 由于闪存存在“写前必须先擦除、擦除以块为单位而读写以页为单位”的物理约束，SSD 必须依赖 FTL 实现逻辑地址到物理地址的映射及异地更新（Out-of-place update）
C. 动态磨损均衡（Dynamic Wear Leveling）算法完全由操作系统内核的文件系统层直接管理，无需 SSD 硬件控制器或 FTL 参与
D. 垃圾回收（Garbage Collection）机制只会在系统断电重启时触发，用于清理无效的闪存块以防止空间碎片化

【答案】： B
【解析】：
A 项错误：闪存不支持原位修改，必须先擦除整块才能写入，因此随机写入性能通常远低于随机读性能
B 项正确：由于闪存“擦除块远大于读写页”的硬件物理约束，SSD 必须依靠闪存转换层（FTL）进行地址转换、逻辑块映射和异地更新
C 项错误：磨损均衡（Wear Leveling）的核心管理和实现逻辑是由 SSD 内部的 FTL 固件和硬件控制器 独立完成的，而不是由操作系统文件系统层管理
D 项错误：垃圾回收（GC）是 SSD 运行过程中的常态化后台动态机制，而非仅在断电重启时触发

30. 【考点：假脱机技术】假脱机技术（SPOOLing）是现代操作系统中实现独占设备共享的核心技术。关于 SPOOLing 技术与 I/O 应用程序接口、设备分配机制的深度融合，下列说法中错误的是（ ）

A. 引入 SPOOLing 技术后，用户进程在向物理打印机输出数据时，实际上是将数据写入磁盘上的“输出井”中，从而将一台独占物理设备虚拟化为多台可供并发使用的逻辑打印机
B. 使用 SPOOLing 技术时，负责将输出井中的数据传送到物理打印机输出的“SPOOLing 进程（守护进程）”通常由系统在后台以低优先级进程方式运行
C. 若用户进程采用非阻塞 I/O 接口向 SPOOLing 输出井提交打印请求，当输出井空间暂时爆满时，系统会强制将该用户进程直接挂起并投入等待队列
D. SPOOLing 技术不仅依赖多道程序设计技术，还必须借助磁盘等共享存储设备来暂存输入/输出数据，从而实现对脱机 I/O 的在线模拟

【答案】： C
【解析】：
A、B、D 项正确：准确概括了 SPOOLing 的空间分配原则（磁盘作井）、虚拟设备分配机制以及后台守护进程的作用

C 项错误：此题考察非阻塞 I/O 的核心特征。非阻塞 I/O 调用的原则是：若操作无法立即完成（如输出井爆满），系统绝不会将进程阻塞挂起，而是会立即返回一个错误码（如 `-EAGAIN`）由用户进程自行决定如何处理；只有阻塞 I/O 才会将进程挂起等待资源